VOLUME 2 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 2

VARIABLES, DATA TYPES & OPERATORS

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 2

About This Module

Welcome to Volume 2 of the IT Study Hub Complete Java Programming Series. In Module 1, you learned how to set up your environment, write a basic Java program, and print text to the screen. However, real-world applications don't just print static text—they store data, manipulate it, and calculate results.

This module breathes life into your programs. You will learn how to allocate memory, understand the exact size and limits of different data forms, and use mathematical and logical operators to build your very first interactive software: a Basic Java Calculator.

Learning Outcomes

- Understand memory allocation through variable declaration and initialization.
- Master all 8 primitive data types (byte, short, int, long, float, double, char, boolean).
- Distinguish between primitives and non-primitives.
- Perform seamless implicit and explicit Type Casting safely.
- Utilize Arithmetic, Relational, Logical, Assignment, Unary, Bitwise, and Ternary operators.
- Build interactive applications using the `Scanner` class to accept user input.

Module Coverage

In this comprehensive guide, we cover: Variables, Constants, the 8 Primitive Data Types, Non-primitives, Literals, Type Casting (Widening and Narrowing), all Operator types, and Operator Precedence. We conclude with a guided hands-on

"Basic Java Calculator" project, followed by 45+ questions (practice, viva, and interview) to test your conceptual clarity.

IT STUDY HUB

1. Variables and Constants

IT STUDY HUB

Concept: Variables (Declaration and Initialization)

> **Simple Explanation:**
> Imagine you are moving houses and you have a bunch of cardboard boxes. You label one box 'Books' and another 'Clothes'. In Java, the memory in your computer is like a giant warehouse. A variable is simply a labeled box in that warehouse where you keep your data. 'Declaration' is getting an empty box and putting a label on it. 'Initialization' is putting something inside that box for the first time.

Syntax / Structure:

```
dataType variableName;          // Declaration
variableName = value;           // Initialization

dataType variableName = value;  // Both in one line
```

Example Code:

```
public class VariableExample {
    public static void main(String[] args) {
        // Declaration and Initialization
        int studentAge = 20;

        // Re-assigning a new value (Variables can change!)
        studentAge = 21;

        System.out.println("The student's age is: " + studentAge);
    }
}
```

Expected Output:

```
The student's age is: 21
```

Code Explanation:

We create a box named `studentAge` meant only for integers (`int`). We put `20` inside it. In the next line, we throw away the `20` and put `21` inside it. When we print, it outputs the current contents of the box.

Real-world Example:
When you play a video game, your score constantly updates. That score is stored in a variable (e.g., `currentScore`).

Common Mistakes:

- Trying to use a variable before declaring it or putting a value inside it.
- Putting the wrong type of item in the box (e.g., trying to store text in an `int` box).

Best Practices:

- Always use meaningful camelCase names (e.g., `accountBalance` instead of `x`).
- Initialize variables as soon as you declare them to avoid unexpected bugs.

Practice Task:
Declare three variables to store your age, your shoe size, and the current year. Print them all out.

IT STUDY HUB

Concept: Constants (The 'final' Keyword)

Simple Explanation:
If a variable is a cardboard box, a constant is a steel safe. Once you put something inside and lock it, you can look at it, but you can never replace it.

Syntax / Structure:

```
final dataType CONSTANT_NAME = value;
```

Example Code:

```
public class ConstantExample {
    public static void main(String[] args) {
        final double PI_VALUE = 3.14159;

        // PI_VALUE = 3.14; // ERROR: Cannot reassign a final variable!

        System.out.println("The value of PI is: " + PI_VALUE);
    }
}
```

Expected Output:

```
The value of PI is: 3.14159
```

Code Explanation:
The `final` keyword locks the variable. Any attempt to change `PI_VALUE` later in the code will cause a compile-time error.

Real-world Example:
Storing the maximum number of login attempts allowed (e.g., 3). You don't want a hacker to accidentally or maliciously change that limit in the program's memory.

Common Mistakes:

- Forgetting to initialize a `final` variable.
- Not using UPPER_SNAKE_CASE naming convention, making it hard for other developers to realize it's a constant.

Best Practices:

- Use constants for values that represent fixed universal rules or configuration limits in your application.

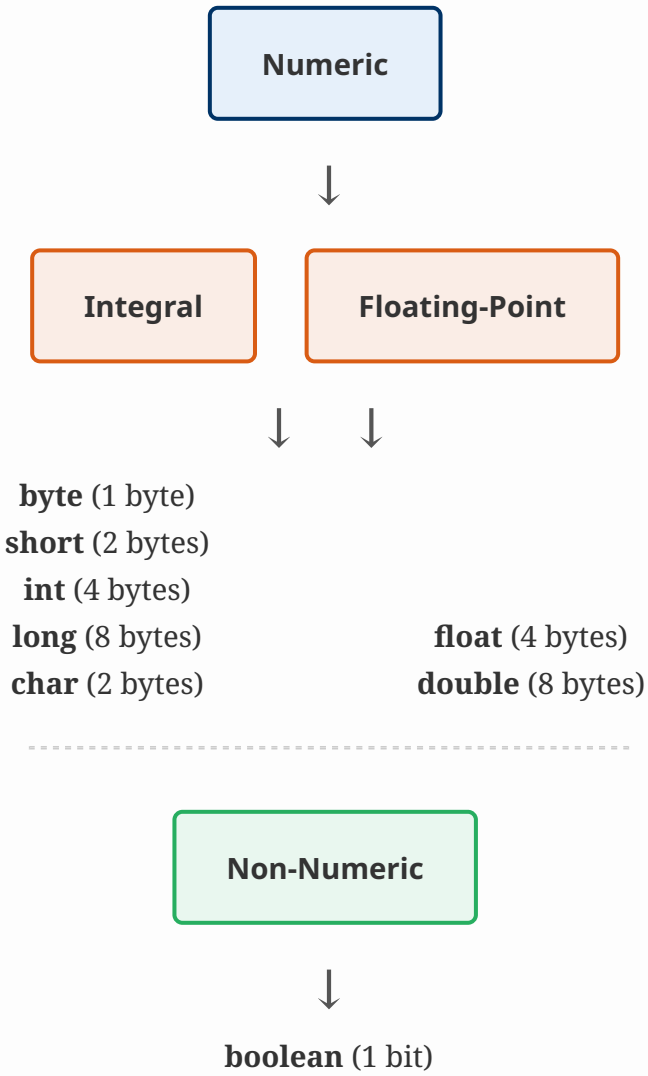
Practice Task:
Declare a constant for the number of days in a week and print it.

IT STUDY HUB

2. Primitive Data Types

Java is a strictly typed language. Every box (variable) must have a specific shape, meaning it can only hold a specific type of data. There are 8 foundational (primitive) data types built directly into Java's memory system.

Java Primitive Data Types Hierarchy



Data Type Size and Range Comparison

Data Type	Size	Default Value	Range of Values
byte	1 byte (8 bits)	0	-128 to 127
short	2 bytes (16 bits)	0	-32,768 to 32,767
int	4 bytes (32 bits)	0	-2^31 to (2^31 - 1) *(~2.1 Billion)*
long	8 bytes (64 bits)	0L	-2^63 to (2^63 - 1) *(Huge!)*
float	4 bytes	0.0f	6 to 7 decimal digits
double	8 bytes	0.0d	15 decimal digits
char	2 bytes	'\u0000'	Single 16-bit Unicode character
boolean	1 bit	false	true or false

Note: We use 2 bytes for 'char' in Java instead of 1 byte (like C++) because Java uses the Unicode system, allowing it to store international characters (like Hindi, Chinese, or Arabic) instead of just English ASCII.

IT STUDY HUB

Concept: Integer Types (byte, short, int, long)

> **Simple Explanation:**
> Why have 4 different types for whole numbers? Efficiency. You don't use a massive shipping container to carry a single apple. If you are storing a person's age (max 120), a 'byte' is enough. If you are storing the population of a country, you need an 'int'.

Syntax / Structure:

```
byte b = 10;
short s = 1000;
int i = 100000;
long l = 10000000000L; // Note the 'L'
```

Example Code:

```
public class IntegerTypes {
    public static void main(String[] args) {
        byte age = 22;
        int distanceToMoon = 384400; // in km
        long globalPopulation = 8100000000L; // Requires L suffix

        System.out.println("Age: " + age);
        System.out.println("Population: " + globalPopulation);
    }
}
```

Expected Output:

```
Age: 22
Population: 8100000000
```

Code Explanation:
Notice the `L` at the end of the `long` value. By default, Java treats all whole numbers written in code as `int`. If the number is too big for an `int`, we must append `L` to tell Java it is a `long`.

> ***Real-world Example:***
> *`int` is the default go-to for almost all whole numbers. `byte` is used in file streaming and network data transfer where every single bit of memory matters.*

Common Mistakes:

- Forgetting the 'L' on a long variable.
- Assigning a value like 130 to a `byte`, which will cause an error because it exceeds the maximum limit of 127.

Best Practices:

- Default to using `int` for whole numbers unless you have a specific memory constraint or the number exceeds 2.1 billion (then use `long`).

Practice Task:
Declare an int for your city's population and a long for the distance from Earth to the Sun.

IT STUDY HUB

Concept: Floating-Point Types (float, double)

Simple Explanation:
When you need precision, like calculating currency or scientific measurements, integers won't work. `float` provides standard precision, while `double` provides double the precision (more decimal places).

Syntax / Structure:

```
float price = 19.99f;  // Note the 'f'
double pi = 3.1415926535;
```

Example Code:

```
public class FloatDouble {
    public static void main(String[] args) {
        float temperature = 36.5f; // Requires 'f' suffix
        double preciseWeight = 75.123456789; // Default decimal type

        System.out.println("Temp: " + temperature);
        System.out.println("Weight: " + preciseWeight);
    }
}
```

Expected Output:

```
Temp: 36.5
Weight: 75.123456789
```

Code Explanation:
By default, Java treats all decimal numbers as `double` because precision is prioritized. If you specifically want to use a `float` (to save memory), you MUST append an `f` or `F`.

Real-world Example:
Calculating GPS coordinates requires extreme precision, hence `double` is used heavily in navigation apps.

Common Mistakes:

- Forgetting the 'f' on a float, causing a 'possible loss of precision' compiler error.

- Using float/double for exact currency calculations. (In advanced Java, we use BigDecimal for money to avoid rounding errors).

Best Practices:

- Default to `double` for decimal numbers unless you are storing millions of decimals in a massive array and need to conserve RAM.

Practice Task:
Declare a double storing the value of gravity (9.81).

IT STUDY HUB

Concept: Characters and Booleans (char, boolean)

> **Simple Explanation:**
> A `char` is just a single letter, number, or symbol enclosed in single quotes. A `boolean` is a digital switch that is either ON (true) or OFF (false).

Syntax / Structure:

```
char grade = 'A'; // Single quotes!
boolean isJavaFun = true;
```

Example Code:

```
public class CharBoolean {
    public static void main(String[] args) {
        char section = 'B';
        char rupeeSymbol = '\u20B9'; // Unicode for Rupee

        boolean isEnrolled = true;
        boolean hasFailed = false;

        System.out.println("Section: " + section);
        System.out.println("Currency: " + rupeeSymbol);
        System.out.println("Enrolled Status: " + isEnrolled);
    }
}
```

Expected Output:

```
Section: B
Currency: ₹
Enrolled Status: true
```

Code Explanation:
Notice how `char` requires single quotes `'`, not double quotes `"`. `boolean` can only ever hold the exact keywords `true` or `false` (all lowercase).

> ***Real-world Example:***
> *`char` is used for processing individual keystrokes in a game. `boolean` is used for conditional logic (e.g., `isLoggedIn = true` gives a user access to their dashboard).*

Common Mistakes:

- Using double quotes for a char (e.g., `char x = "A";`). Double quotes are for Strings!
- Trying to assign 1 or 0 to a boolean. In C++, 1 is true, but in Java, it strictly requires the word `true`.

Best Practices:

- Use clearly understandable boolean names starting with 'is' or 'has' (e.g., `isValid`, `hasPermission`).

Practice Task:
Declare a boolean representing whether it is raining right now, and a char representing your favorite letter.

IT STUDY HUB

3. Non-Primitive Data Types (Reference Types)

Unlike primitives which store the actual raw value, **Non-Primitive** (or Reference) data types store the *memory address* where the complex data is held. They are objects.

- **String:** A sequence of characters (e.g., "Hello World"). Note the capital 'S'.
- **Arrays:** A collection of similar types of data.
- **Classes / Interfaces:** Custom data types defined by the user.

```
public class NonPrimitives {
    public static void main(String[] args) {
        // String is a non-primitive. It uses double quotes!
        String studentName = "Alice";
        System.out.println("Name: " + studentName);
    }
}
```

We will dive deep into Strings and Arrays in Module 4. For now, just know that `String` requires a capital 'S' and uses double quotes.

IT STUDY HUB

4. Literals in Java

A literal is simply the raw, fixed value that you type directly into the source code to assign to a variable.

Type of Literal	Examples	Description
Integer Literal	`100`, `-50`, `0b1010` (binary)	Whole numbers typed directly.
Floating-point Literal	`3.14`, `-0.001`, `1.5e3` (scientific)	Decimals typed directly.
Character Literal	`'A'`, `'9'`, `'\n'` (newline)	Single characters in single quotes.
String Literal	`"Hello"`, `"IT Study Hub"`	Text in double quotes.
Boolean Literal	`true`, `false`	The specific logical keywords.

IT STUDY HUB

5. Type Casting

IT STUDY HUB

Concept: Implicit & Explicit Conversion

> **Simple Explanation:**
> Imagine you have a small cup of water (int) and a large bucket (double). Pouring the cup into the bucket is easy and safe—this is Implicit Casting (Widening). Now imagine trying to pour the large bucket into the small cup. The water might spill! Java won't let you do this unless you explicitly force it, acknowledging the risk of data loss. This is Explicit Casting (Narrowing).

Syntax / Structure:

```
// Widening (Automatic)
double bigContainer = smallInteger;

// Narrowing (Manual - Requires (targetType))
int smallContainer = (int) largeDouble;
```

Example Code:

```
public class TypeCasting {
    public static void main(String[] args) {
        // 1. Implicit Casting (Widening): int -> double
        int myInt = 9;
        double myDouble = myInt; // Automatic

        System.out.println("Original int: " + myInt);
        System.out.println("Widened double: " + myDouble);

        // 2. Explicit Casting (Narrowing): double -> int
        double exactPrice = 99.99;
        int roundedPrice = (int) exactPrice; // Manual cast

        System.out.println("\nOriginal double: " + exactPrice);
        System.out.println("Narrowed int: " + roundedPrice);
        // Notice the .99 is completely lost!
    }
}
```

Expected Output:

```
Original int: 9
Widened double: 9.0
```

```
Original double: 99.99
Narrowed int: 99
```

Code Explanation:
When we force `99.99` into an `int`, Java doesn't round to the nearest whole number. It just violently chops off (truncates) the decimal part entirely. That's why we get `99`, not `100`.

Real-world Example:
If a user inputs their weight as 75.8kg in a form, but your database only accepts whole integers, you cast it to `(int)` to store 75.

Common Mistakes:

- Trying to narrow a data type without using the `(type)` syntax. The compiler will block you with an 'incompatible types' error.

Best Practices:

- Be very careful with explicit casting. Always verify if the loss of decimal precision will break your application's logic (especially in finance).

Practice Task:
Declare a double with value 15.7. Cast it to an int and print both values.

Widening Conversion (Automatic / Safe)
byte → short → char → int → long → float → double

Narrowing Conversion (Manual / Risky)
double → float → long → int → char → short → byte

---

6. Arithmetic and Assignment Operators

Operators are special symbols that perform specific operations on one, two, or three operands (variables or values), and then return a result.

IT STUDY HUB

Concept: Arithmetic Operators

> **Simple Explanation:**
> These do exactly what you learned in elementary math. The only new one is the Modulus operator (`%`), which divides two numbers and returns the **remainder**, not the quotient.

Syntax / Structure:

```
result = a + b;
remainder = a % b;
```

Example Code:

```
public class Arithmetic {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;

        System.out.println("Addition: " + (a + b));      // 13
        System.out.println("Division: " + (a / b));      // 3
        System.out.println("Modulus: " + (a % b));       // 1
    }
}
```

Expected Output:

```
Addition: 13
Division: 3
Modulus: 1
```

Code Explanation:
Notice `10 / 3` results in `3`, not `3.333`. Why? Because both `10` and `3` are `int`. Integer division cuts off the decimal. The Modulus `10 % 3` returns `1` (because 3 goes into 10 three times, leaving a remainder of 1).

> ***Real-world Example:***
> *Modulus `%` is incredibly useful. We use `number % 2 == 0` to check if a number is even or odd!*

Common Mistakes:

- Thinking `/` will give decimals when dividing integers.
- Confusing `%` (remainder) with percentage.

Best Practices:

- If you need decimal division, make sure at least one number is a double (e.g., `10.0 / 3`).

Practice Task:
Write code that divides 25 by 4, prints the quotient, and then prints the remainder using %.

IT STUDY HUB

Concept: Assignment Operators

> **Simple Explanation:**
> Instead of writing `score = score + 5;`, programmers use a shorthand operator: `score += 5;`. It means 'take the current box, add 5 to whatever is inside, and save it back'.

Syntax / Structure:

```
x += y;   // equivalent to: x = x + y;
```

Example Code:

```
public class Assignment {
    public static void main(String[] args) {
        int health = 100;

        // Player takes 20 damage
        health -= 20; // health = health - 20;
        System.out.println("Health after attack: " + health);

        // Player finds a double-health potion
        health *= 2;  // health = health * 2;
        System.out.println("Health after potion: " + health);
    }
}
```

Expected Output:

```
Health after attack: 80
Health after potion: 160
```

Code Explanation:
The compound operators cleanly update the variable in place without repeating the variable name.

> ***Real-world Example:***
> *Used everywhere in loop counters, score trackers, and physics engines to update object positions.*

Common Mistakes:

- Writing `=+` instead of `+=`. `x =+ 5` just means `x = positive 5`.

Best Practices:

- Always use compound assignment operators to make your code cleaner and more professional.

Practice Task:

Initialize an int to 10. Multiply it by 5 using a compound operator, then print it.

IT STUDY HUB

7. Relational and Logical Operators

IT STUDY HUB

Concept: Relational (Comparison) Operators

> **Simple Explanation:**
> You are asking a question. 'Is X greater than Y?' Java answers with true or false.

Syntax / Structure:

```
boolean result = (a == b);
```

Example Code:

```
public class Relational {
    public static void main(String[] args) {
        int passingMarks = 40;
        int myMarks = 45;

        boolean hasPassed = myMarks >= passingMarks;
        boolean isTopper = myMarks == 100;

        System.out.println("Passed? " + hasPassed);
        System.out.println("Topper? " + isTopper);
    }
}
```

Expected Output:

```
Passed? true
Topper? false
```

Code Explanation:
`myMarks >= passingMarks` evaluates to `45 >= 40`, which is a true statement. So, `true` is stored in the boolean box.

> ***Real-world Example:***
> *Determining if a user is old enough to access a website: `age >= 18`.*

> **Common Mistakes:**
>
> - Using a single `=` for comparison. Single `=` is ASSIGNMENT. Double `==` is COMPARISON. This is the #1 beginner mistake!

Best Practices:

- Wrap comparisons in parentheses to make code readable: `boolean isAdult = (age >= 18);`

Practice Task:
Declare two integers. Print true if they are NOT equal using the != operator.

IT STUDY HUB

Concept: Logical Operators

> **Simple Explanation:**
> AND (`&&`): BOTH conditions must be true for the result to be true. OR (`||`): AT LEAST ONE condition must be true. NOT (`!`): Flips the result (true becomes false).

Syntax / Structure:

```
if (condition1 && condition2)
```

Example Code:

```
public class Logical {
    public static void main(String[] args) {
        boolean hasTicket = true;
        boolean hasID = false;

        // AND: Needs both
        boolean canEnterMovie = (hasTicket && hasID);

        // OR: Needs only one
        boolean hasCreditCard = true;
        boolean hasCash = false;
        boolean canBuyFood = (hasCreditCard || hasCash);

        // NOT: Reverses
        boolean isBanned = false;

        System.out.println("Can enter movie? " + canEnterMovie);
        System.out.println("Can buy food? " + canBuyFood);
        System.out.println("Is allowed inside? " + !isBanned);
    }
}
```

Expected Output:

```
Can enter movie? false
Can buy food? true
Is allowed inside? true
```

Code Explanation:
For the movie, `true && false` evaluates to `false`. For food, `true || false` evaluates to `true`. Finally, `!false` evaluates to `true`.

> ***Real-world Example:***
> *Validating a login screen requires BOTH the correct username AND the correct password: `(isValidUser && isValidPass)`.*

> **Common Mistakes:**
>
> - Using single `&` or `|`. In Java, single ampersand/pipe perform Bitwise operations, not short-circuit Logical operations.

> **Best Practices:**
>
> - Rely on 'Short-Circuiting'. In `(A && B)`, if A is false, Java doesn't even bother checking B because it knows the whole statement will be false. This saves processing power!

Practice Task:
Create boolean variables for 'isWeekend' and 'isHoliday'. Use an OR operator to print true if you get to sleep in.

---

8. Unary, Increment, Decrement and Bitwise Operators

IT STUDY HUB

Concept: Unary (Increment/Decrement) Operators

> **Simple Explanation:**
> Instead of writing `count = count + 1` or `count += 1`, you can just write `count++`.

Syntax / Structure:

```
x++;  // Post-increment
++x;  // Pre-increment
```

Example Code:

```
public class Unary {
    public static void main(String[] args) {
        int x = 5;
        int y = 5;

        System.out.println("Post-increment (x++): " + (x++));
        System.out.println("Value of x after: " + x);

        System.out.println("\nPre-increment (++y): " + (++y));
        System.out.println("Value of y after: " + y);
    }
}
```

Expected Output:

```
Post-increment (x++): 5
Value of x after: 6

Pre-increment (++y): 6
Value of y after: 6
```

Code Explanation:
Crucial difference: `x++` (post) uses the original value (5) for the print statement first, and THEN increments it to 6 in memory. `++y` (pre) increments the value to 6 in memory first, and THEN uses it for the print statement.

> ***Real-world Example:***
> *Used constantly in loop counters (e.g., running through a list of 100 students one by one).*

Common Mistakes:

- Getting confused between pre and post increment when assigning to a new variable (e.g., `a = b++`).

Best Practices:

- Avoid complex chains like `z = x++ + ++y;`. It is unreadable and leads to bugs. Keep increments on their own dedicated lines.

Practice Task:
Set an int to 10. Decrement it using post-decrement on a separate line, then print it.

Bitwise Operators (Brief Overview)

Bitwise operators (&, |, ^, ~, <<, >>) perform operations directly on the binary bits (1s and 0s) of integers.

- They are highly advanced and rarely used in standard enterprise applications.
- They are used heavily in low-level systems programming, cryptography, and image processing where speed and bit-level memory manipulation are critical.

Example: `5 & 3`. In binary, 5 is `0101` and 3 is `0011`. A Bitwise AND compares each bit vertically, returning `0001` (which is 1 in decimal).

---

9. Ternary Operator and Operator Precedence

IT STUDY HUB

Concept: The Ternary (Conditional) Operator

Simple Explanation:
It's a one-line question. Condition ? (Value if True) : (Value if False).

Syntax / Structure:

```
variable = (condition) ? valueTrue : valueFalse;
```

Example Code:

```
public class Ternary {
    public static void main(String[] args) {
        int age = 20;

        // If age >= 18 is true, store "Adult", else store "Minor"
        String status = (age >= 18) ? "Adult" : "Minor";

        System.out.println("Status: " + status);
    }
}
```

Expected Output:

```
Status: Adult
```

Code Explanation:
Java evaluates `age >= 18`. Since 20 is greater than 18, it picks the first option before the colon (`"Adult"`). If age was 15, it would pick the option after the colon (`"Minor"`).

Real-world Example:
Used to quickly format output text based on a status flag without writing 5 lines of clunky if-else code.

Common Mistakes:

- Trying to use ternary operators for massive blocks of code. It should only be used for simple variable assignment.

Best Practices:

- Always wrap the condition in parentheses for readability.

Practice Task:
Use a ternary operator to assign the String 'Even' or 'Odd' to a variable based on whether a number % 2 is 0.

Operator Precedence (BODMAS for Java)

When an expression contains multiple operators, Java evaluates them based on a strict hierarchy, just like BODMAS/PEMDAS in math.

Precedence (Highest to Lowest)	Operator Types	Associativity
1 (Highest)	Postfix `expr++` `expr--`	Left to Right
2	Prefix `++expr` `--expr`, Unary `+` `-` `!`	Right to Left
3	Multiplicative `*` `/` `%`	Left to Right
4	Additive `+` `-`	Left to Right
5	Relational `<` `>` `<=` `>=`	Left to Right
6	Equality `==` `!=`	Left to Right
7	Logical AND &&	Left to Right
8	Logical OR ||	Left to Right
9	Ternary `?` `:`	Right to Left
10 (Lowest)	Assignment `=` `+=` `-=` `*=` `/=` `%=`	Right to Left

Example: In `int x = 5 + 3 * 2;`, multiplication is higher than addition. 3*2 evaluates first (6), then 5+6 (11). To force addition first, use parentheses: `(5 + 3) * 2 = 16`.

IT STUDY HUB

10. Project: Basic Java Calculator

Let's combine everything we've learned—variables, data types, operators, and a new concept: **User Input**. To get input from the keyboard, we must import and use the `Scanner` class provided by Java.

```
import java.util.Scanner;  // 1. Import the Scanner tool

public class BasicCalculator {
    public static void main(String[] args) {

        // 2. Create a Scanner object to read from the console (System.in)
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== BASIC JAVA CALCULATOR ===");

        // 3. Prompt user and read first double
        System.out.print("Enter the first number: ");
        double num1 = scanner.nextDouble();

        // 4. Prompt user and read second double
        System.out.print("Enter the second number: ");
        double num2 = scanner.nextDouble();

        // 5. Perform Operations using Arithmetic Operators
        double sum = num1 + num2;
        double difference = num1 - num2;
        double product = num1 * num2;

        // 6. Output the formatted results
        System.out.println("\n--- RESULTS ---");
        System.out.println("Addition (+): " + sum);
        System.out.println("Subtraction (-): " + difference);
        System.out.println("Multiplication (*): " + product);

        // 7. Validation logic using Relational/Ternary operators
        String divResult = (num2 == 0) ? "Error: Cannot divide by zero!" : String.valueO
        System.out.println("Division (/): " + divResult);

        // 8. Close the scanner to free up memory
```

```
        scanner.close();
        System.out.println("============================");
    }
}
```

```
=== BASIC JAVA CALCULATOR ===
Enter the first number: 15.5
Enter the second number: 4

--- RESULTS ---
Addition (+): 19.5
Subtraction (-): 11.5
Multiplication (*): 62.0
Division (/): 3.875
============================
```

Step-by-Step Explanation:

1. **import java.util.Scanner:** Java has thousands of built-in tools. We tell Java we want to bring the Scanner tool into our program.
2. **new Scanner(System.in):** We initialize the Scanner, pointing it at the standard input stream (your keyboard).
3. **nextDouble():** This command pauses the program, waits for the user to type a decimal number, and press Enter. It stores that value in `num1`.
4. **Operations:** We create new variables to store the mathematical results. We use `double` so the calculator handles decimals accurately.
5. **Validation (Ternary):** If the user enters `0` for `num2`, dividing by zero breaks mathematics and crashes programs. We use a ternary operator to check: *is num2 exactly equal to 0?* If true, print an error message. If false, perform the division.
6. **close():** Best practice. Once we are done taking inputs, we shut down the Scanner to prevent memory leaks.

---

11. Module Assessment and Interview Prep

This section is designed to test your knowledge, prepare you for university vivas, and give you an edge in junior developer interviews.

A. Practice Questions (Test Your Logic)

1. Declare variables of appropriate primitive types to store a person's age, exact bank balance, initial of their first name, and whether they are married.
2. Write a program that swaps the values of two integer variables.
3. What will `System.out.println(10 + 20 + "Hello" + 10 + 20);` output? Why?
4. Declare a final variable for the speed of light. Try to modify it on the next line and observe the compiler error.
5. Write a program that takes an integer input from the user using Scanner and prints whether it is even or odd using the modulo operator.
6. Perform explicit type casting to convert `double d = 100.99;` to an `int`. Print the result.
7. What is the difference in output between `int x=5; System.out.print(x++);` and `int y=5; System.out.print(++y);`?
8. Evaluate this expression by hand using precedence rules: `int res = 10 * 2 + 5 % 2 - 1;`
9. Write a program to calculate the area of a circle. (Area = π * r * r). Take radius as user input.
10. Using the ternary operator, write code that assigns 'Pass' or 'Fail' to a String variable based on whether an int `score` is greater than 50.
11. What happens if you assign the value `300` to a `byte` variable?
12. Create a program that converts Celsius to Fahrenheit. Formula: (C * 9/5) + 32.
13. Demonstrate short-circuiting by writing an `if` condition using `&&` where the first condition is `false`. Print a message to prove the second condition was skipped.
14. Combine compound assignment and mathematical operators: Initialize a variable to 10, multiply it by 3 using a compound operator, and print.
15. Explain what the literal `0b1010` represents in Java.

IT STUDY HUB

B. University Viva Questions & Answers

Q: What is a variable?

Ans: A variable is a named memory location used to store data that can be modified during program execution.

Q: What are primitive data types?

Ans: Built-in data types that hold standard raw values directly in memory. There are 8 in Java: byte, short, int, long, float, double, char, boolean.

Q: What is the size and default value of an int?

Ans: Size is 4 bytes (32 bits). Default value is 0.

Q: Why does char take 2 bytes in Java but 1 byte in C/C++?

Ans: Because Java uses the Unicode character set to support international characters, whereas C/C++ traditionally uses the ASCII set.

Q: What is a literal?

Ans: A literal is a fixed value directly written into the source code, like 10, 3.14, 'A', or true.

Q: What is the difference between float and double?

Ans: Float is single-precision (4 bytes) and requires an 'f' suffix. Double is double-precision (8 bytes), the default for decimals, and provides higher mathematical accuracy.

Q: What is Type Casting?

Ans: The process of converting a variable of one data type into another.

Q: Explain implicit casting.

Ans: Also known as Widening. It happens automatically when transferring a smaller data type into a larger one (e.g., int to double). There is no loss of data.

Q: Explain explicit casting.

Ans: Also known as Narrowing. It requires manual casting via (type) when moving a larger type into a smaller type (e.g., double to int). It can result in data loss.

Q: What does the 'final' keyword do to a variable?

Ans: It makes the variable a constant. Once initialized, its value can never be changed.

IT STUDY HUB

Q: What is the Modulus operator (%)?

Ans: It divides two numbers and returns the remainder of the division.

Q: What is the difference between = and ==?

Ans: Single = is the assignment operator (stores a value in a variable). Double == is a relational operator (compares two values for equality).

Q: What does the ++ operator do?

Ans: It is the increment operator. It increases the value of a numeric variable by exactly 1.

Q: What is the difference between pre-increment and post-increment?

Ans: Pre-increment (++x) increments the value immediately before using it in the current statement. Post-increment (x++) uses the current value in the statement, then increments it afterwards.

Q: What are Logical Operators?

Ans: Operators (&&, ||, !) used to combine or invert boolean expressions.

Q: What is short-circuit evaluation?

Ans: In a logical AND (&&), if the first operand is false, the second is not evaluated because the overall result must be false. In a logical OR (||), if the first is true, the second is skipped.

Q: What is the Ternary operator?

Ans: A one-line conditional operator that takes three operands: a boolean condition, a value if true, and a value if false. (condition ? true_val : false_val).

Q: Why must we close the Scanner object?

Ans: To release the system resources and input streams, preventing memory leaks in large applications.

Q: Is String a primitive data type?

Ans: No, String is a Non-Primitive (reference) type representing a sequence of characters. It is fundamentally an object in Java.

Q: What is Operator Precedence?

Ans: The rules that determine the order in which different operators are evaluated in a complex expression (e.g., multiplication before addition).

IT STUDY HUB

C. Beginner Interview Questions

Interview Q: Why is Java strongly typed?

Ans: Java requires every variable to be declared with a specific data type before it can be used, and the compiler strictly enforces that the stored data matches this type. This prevents unexpected type-related crashes at runtime.

Interview Q: What happens if an integer exceeds its maximum value limit (overflow)?

Ans: Java does not throw an exception. Instead, it silently 'wraps around' to the minimum negative value for that data type. This is why choosing the correct data type (int vs long) is crucial.

Interview Q: Explain the difference between primitive and reference (non-primitive) types in memory.

Ans: Primitive types store the actual physical value directly in the memory location associated with the variable (on the Stack). Reference types store the memory address (reference/pointer) of where the actual complex object is located (on the Heap).

Interview Q: If you divide 5 by 2 in Java, what is the output and why?

Ans: The output is 2, not 2.5. Because both 5 and 2 are integers, Java performs integer division, which truncates the decimal portion. To get 2.5, you must make at least one operand a float/double (e.g., 5.0 / 2).

Interview Q: Can we perform arithmetic operations on char types?

Ans: Yes. Characters in Java are internally mapped to integer Unicode values (e.g., 'A' is 65). You can add an integer to a char to get the next character in the sequence (e.g., 'A' + 1 = 'B').

Interview Q: What is the use of bitwise operators in modern development?

Ans: While rare in high-level business logic, they are essential in low-level programming, network packet parsing, hardware interaction, and performance optimization where manipulating individual bits is necessary.

Interview Q: Why should we avoid floating-point types for currency?

Ans: Floats and doubles store approximations of decimals using binary fractions, which can lead to tiny precision errors (e.g., 0.1 + 0.2 returning 0.30000000000000004). For exact financial calculations, Java provides the BigDecimal class.

IT STUDY HUB

Interview Q: How does the ternary operator improve code quality?

Ans: It reduces boilerplate code. Instead of writing a 5-line if-else block for a simple assignment, it condenses the logic into a single, highly readable line, making the code cleaner and easier to maintain.

Interview Q: Is boolean guaranteed to take 1 bit of memory?

Ans: While it logically represents 1 bit of information (true/false), its actual size in memory is JVM-dependent. Most virtual machines pad it to 1 byte for memory alignment efficiency.

Interview Q: What error will you get if you use a variable before initializing it?

Ans: A compile-time error: 'variable [name] might not have been initialized'. Java enforces initialization to ensure programs don't output unpredictable garbage data from memory.

IT STUDY HUB

12. Module Summary and Key Terms

Key Terms Glossary

- **Initialization:** The act of assigning a value to a variable for the very first time.
- **Primitive:** The 8 core, irreducible data types built into the Java language.
- **Literal:** A raw value explicitly typed in code (e.g., 100, "Text", true).
- **Casting:** Forcing a value of one data type into a variable of another data type.
- **Truncation:** The process of chopping off the decimal part of a number (not rounding) when casting from double to int.
- **Operand:** The variables or values that an operator acts upon (e.g., in x + y, x and y are operands).

Module Summary

In this module, you transformed from writing static text to managing live data in memory. You learned how to declare variables and lock them as constants using the `final` keyword. You mastered the 8 primitive data types, understanding exactly how much memory each occupies and when to use them.

We explored how data can morph via Implicit and Explicit Type Casting, and the dangers of data loss when narrowing. You then gained the power to manipulate this data using a full suite of Operators—performing mathematics, evaluating logical conditions, and updating values concisely.

Finally, we brought it all together by introducing the `Scanner` class, allowing your programs to listen to the user and dynamically calculate results in the Basic Java Calculator project.

WHAT COMES NEXT?

Module 3 — Control Flow & Methods

Our calculator works, but what if we want to run it repeatedly without restarting the program? What if we want to give the user a menu of options? In the next volume, you will master conditional logic (if-else, switch) to make your code intelligent, and loops (for, while, do-while) to make it automated. We will also learn how to break complex programs into reusable 'Methods'.

IT STUDY HUB SERIES